

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



GRAPH DAN TREE

Oleh:

Muhammad Rifqi Rahman

NIM. 2510817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
TAHUN 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rifqi Rahman
NIM : 2510817310003

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	v
DAFTAR TABEL	vi
SOAL 1.....	1
A. Source Code.....	3
B. Output Program	4
C. Pembahasan	4
SOAL 2.....	6
A. Source Code.....	8
B. Output Program	9
C. Pembahasan	9
SOAL 3.....	11
A. Source Code.....	13
B. Output Program	15
C. Pembahasan	15
SOAL 4.....	17
A. Source Code.....	19
B. Output Program	20
C. Pembahasan	21
SOAL 5.....	23
A. Source Code.....	25
B. Output Program	28

C. Pembahasan	28
TAUTAN GIT	30

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	4
Gambar 2. Screenshot Hasil Jawaban Soal 2	9
Gambar 3. Screenshot Hasil Jawaban Soal 3	15
Gambar 4. Screenshot Hasil Jawaban Soal 4	20
Gambar 5. Screenshot Hasil Jawaban Soal 5	28

DAFTAR TABEL

Tabel 1 Source Code prob1.cpp	3
Tabel 2 Source Code prob2.cpp	8
Tabel 3 Source Code prob3.cpp	13
Tabel 4 Source Code prob4.cpp	19
Tabel 5 Source Code prob5.cpp	25

SOAL 1

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .

- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency Matrix:

A B C D E 6	A B C D E
A B 4	A 0 4 2 0 0
A C 2	B 0 0 0 7 0
B D 7	C 0 1 0 0 6
C B 1	D 0 0 3 0 0
C E 6	E 0 0 0 0 0
D C 3	

A. Source Code

Tabel 1 Source Code probl.cpp

```

1  #include <iostream>
2  #include <map>
3  #include <vector>
4  using namespace std;
5
6  int main() {
7      int N;
8      cin >> N;
9      vector<char> labels(N);
10     map<char, int> idx;
11     for (int i = 0; i < N; i++) {
12         cin >> labels[i];
13         idx[labels[i]] = i;
14     }
15
16     vector<vector<int>> matrix(N, vector<int>(N,
17     0));
18
19     int M;
20     cin >> M;
21     for (int i = 0; i < M; i++) {

```

```

21         char u, v; int w;
22         cin >> u >> v >> w;
23         matrix[idx[u]][idx[v]] = w;
24     }
25
26     cout << "Adjacency Matrix:" << endl;
27     cout << " ";
28     for (int i = 0; i < N; i++) cout << " " <<
labels[i];
29     cout << endl;
30     for (int i = 0; i < N; i++) {
31         cout << labels[i];
32         for (int j = 0; j < N; j++) cout << " "
<< matrix[i][j];
33         cout << endl;
34     }
35     return 0;
36 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\probl.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Pada baris [1, 2, dan 3] syntax `#include` mengimpor library standar C++ untuk input-output, pemetaan data, dan array dinamis bernilai kompleksitas waktu dan ruang $O(1)$, pada baris [4] syntax `using namespace std;` mendeklarasikan ruang nama standar, pada

baris [6 hingga 8] syntax `int main()` serta `cin >> N` menginisiasi program dan membaca jumlah vertex graf berarah berbobot, pada baris [9 dan 10] syntax `vector` dan `map` memesan memori untuk menyimpan urutan label dan memetakan karakter unik ke indeks numerik dengan kerumitan ruang $O(N)$, pada baris [11 hingga 14] syntax `for` dan `cin` menerima elemen label sekaligus menyimpannya ke dalam struktur pemetaan dengan kompleksitas waktu $O(N \log N)$, pada baris [16] syntax `vector<vector<int>> matrix` mengalokasikan memori matriks dua dimensi berukuran $N \times N$ yang diinisialisasi dengan angka 0 yang memakan ruang dan waktu $O(N^2)$, pada baris [18 hingga 24] syntax `cin >> M` beserta perulangan `for` memproses input edge untuk memanipulasi sel matriks secara langsung menggunakan pencarian indeks (`matrix[idx[u]][idx[v]] = w;`) guna mengubah adjacency list menjadi adjacency matrix dengan kompleksitas waktu $O(M \log N)$, pada baris [26 hingga 29] syntax `cout` mencetak header baris pertama matriks secara linier $O(N)$, pada baris [30 hingga 34] perulangan `for` bersarang digunakan untuk menelusuri dan mencetak keseluruhan elemen baris serta kolom adjacency matrix ke layar pengguna yang berjalan dengan kompleksitas waktu $O(N^2)$, lalu pada baris [35] syntax `return 0;` menutup eksekusi fungsi utama secara normal.

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edgenya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

$V_N \rightarrow \dots$

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol $\rightarrow$.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency List:
A B C D E	A $\rightarrow$ (B, 4), (C, 2)
0 4 2 0 0	B $\rightarrow$ (D, 7)
0 0 0 7 0	C $\rightarrow$ (B, 1), (E, 6)
0 1 0 0 6	D $\rightarrow$ (C, 3)
0 0 3 0 0	E $\rightarrow$ -

0	0	0	0	0	
---	---	---	---	---	--

A. Source Code

Tabel 2 Source Code prob2.cpp

```

1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      int N;
7      cin >> N;
8      vector<char> labels(N);
9      for (int i = 0; i < N; i++) cin >> labels[i];
10
11         vector<vector<int>> matrix(N,
vector<int>(N));
12     for (int i = 0; i < N; i++)
13         for (int j = 0; j < N; j++)
14             cin >> matrix[i][j];
15
16     cout << "Adjacency List:" << endl;
17     for (int i = 0; i < N; i++) {
18         cout << labels[i] << " -> ";
19         bool hasEdge = false;
20         for (int j = 0; j < N; j++) {
21             if (matrix[i][j] > 0) {
22                 if (hasEdge) cout << ", ";
23                 cout << "(" << labels[j] << ", "
<< matrix[i][j] << ")";
24                 hasEdge = true;
25             }
26         }

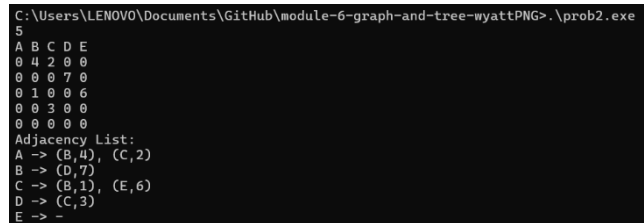
```

```

27         if (!hasEdge) cout << "-";
28         cout << endl;
29     }
30     return 0;
31 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>. \prob2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -

```

Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [5 hingga 7] syntax `int main()` dan `cin >> N` membaca masukan kapasitas ukuran vertex graf, pada baris [8 dan 9] syntax `vector<char> labels` dan perulangan `for` mengalokasikan memori serta mendaftarkan label unik tiap vertex secara linier dengan kompleksitas ruang memori parsial $O(N)$, pada baris [11] syntax `vector<vector<int>> matrix` mendeklarasikan ruang matriks dua dimensi yang menjadi representasi Adjacency Matrix awal sehingga mendominasi kompleksitas ruang algoritma secara keseluruhan sebesar $O(N^2)$, pada baris [12 hingga 14] perulangan `for` bersarang digunakan untuk membaca dan menyalin bilangan input ke sel-sel matriks yang berjalan dengan kompleksitas waktu $O(N^2)$, pada baris [16] syntax `cout` mencetak format teks judul cetakan keluaran, pada baris [17 hingga 29] perulangan `for` bersarang menelusuri ulang seluruh koordinat baris dan kolom matriks sebagai inti algoritma pengonversian graf yang beroperasi dengan kompleksitas waktu dominan $O(N^2)$, pada baris [19] syntax `bool hasEdge = false;` diinisialisasi secara spesifik sebagai pelacak logis atas eksistensi jalur keluar pada vertex tertentu, pada baris [21 hingga 25] syntax `if (matrix[i][j] > 0)` merepresentasikan mekanisme utama algoritma dengan mengevaluasi sel bernilai positif sebagai bobot sisi (edge) yang terhubung ke vertex tujuan untuk kemudian dirangkai ke layar sebagai Adjacency List sekaligus mengubah status indikator `hasEdge` menjadi

true, pada baris [27] syntax *if (!hasEdge)* mengonfirmasi apabila sebuah vertex tidak memiliki rute keluar sama sekali dan meresponsnya dengan mencetak simbol `-`.

SOAL 3

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G11 G12 ... G1C

G21 G22 ... G2C

...

GR1 GR2 ... GRC

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0	16

1 1 1 0 1 0 1 1 1 0	
0 0 1 0 0 0 0 0 1 0	
0 1 1 1 1 1 1 0 1 0	
0 0 0 0 0 0 1 0 0 0	
1 1 1 1 1 0 1 1 1 0	
0 0 0 0 1 0 0 0 1 0	
0 1 1 0 0 0 1 0 0 0	
0 0	
7 9	

A. Source Code

Tabel 3 Source Code prob3.cpp

```

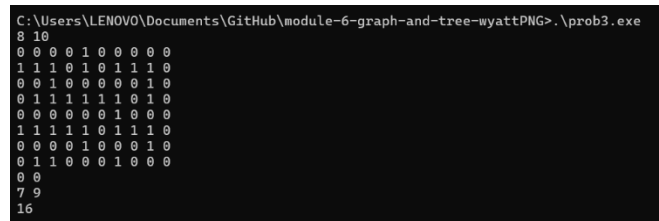
1  #include <iostream>
2  #include <vector>
3  #include <queue>
4  using namespace std;
5
6  int main() {
7      int R, C;
8      cin >> R >> C;
9      vector<vector<int>> > grid(R,
vector<int>(C));
10     for (int i = 0; i < R; i++)
11         for (int j = 0; j < C; j++)
12             cin >> grid[i][j];
13
14     int SR, SC, FR, FC;
15     cin >> SR >> SC >> FR >> FC;
16
17     if (SR == FR && SC == FC) { cout << 0 <<
endl; return 0; }
```

```

18
19     vector<vector<int> > dist(R, vector<int>(C,
20     -1));
21     queue<pair<int,int> > q;
22     dist[SR][SC] = 0;
23     q.push(make_pair(SR, SC));
24
25     int dr[] = {-1, 1, 0, 0};
26     int dc[] = {0, 0, -1, 1};
27
28     while (!q.empty()) {
29         int r = q.front().first;
30         int c = q.front().second;
31         q.pop();
32         for (int d = 0; d < 4; d++) {
33             int nr = r + dr[d], nc = c + dc[d];
34             if (nr >= 0 && nr < R && nc >= 0 &&
35             nc < C && grid[nr][nc] == 0 && dist[nr][nc] ==
36             -1) {
37                 dist[nr][nc] = dist[r][c] + 1;
38                 if (nr == FR && nc == FC) { cout
39                 << dist[nr][nc] << endl; return 0; }
40                 q.push(make_pair(nr, nc));
41             }
42         }
43     }
44
45     cout << -1 << endl;
46     return 0;
47 }

```

B. Output Program



```
C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\prob3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
```

Gambar 3. Screenshot Hasil Jawaban Soal 3

C. Pembahasan

Algoritma ini mengimplementasikan metode Breadth-First Search (BFS) untuk menemukan jumlah langkah minimum rute keluar dari dalam sebuah labirin grid 2D, di mana algoritma akan menelusuri setiap kemungkinan arah yang valid (atas, bawah, kiri, kanan) secara terstruktur tingkat demi tingkat guna menjamin pencapaian jarak terpendek secara mutlak. Pada baris [6 hingga 8] syntax `int main()` dan `cin` memulai eksekusi program serta membaca ukuran dimensi baris dan kolom labirin, pada baris [9 hingga 12] syntax `vector<vector<int>>` grid beserta perulangan `for` memesan ruang memori matriks dan menyalin konfigurasi jalan maupun dinding berdasar input, yang mana mendominasi kompleksitas waktu serta memori ruang algoritma secara keseluruhan menjadi $O(R \times C)$, pada baris [14 dan 15] syntax `cin` membaca rincian koordinat sel awal dan tujuan, pada baris [17] syntax `if` mengevaluasi secara instan $O(1)$ apabila posisi awal dan tujuan telah berada di titik yang identik sejak semula, pada baris [19 dan 20] syntax `vector<vector<int>>` dist bernilai inisialisasi `-1` dan `queue<pair<int,int>>` `q` dialokasikan di dalam memori untuk melacak rekam jejak jarak tempuh sekaligus menampung antrean pergerakan simpul BFS yang turut mereservasi kompleksitas ruang tambahan sebesar $O(R \times C)$, pada baris [21 dan 22] koordinat sel awal disetel dengan jarak `0` dan ditambahkan ke antrean eksekusi queue, pada baris [24 dan 25] deret array numerik `dr` dan `dc` dideklarasikan untuk memetakan empat arah pergerakan ortogonal secara konstan, pada baris [27 hingga 39] perulangan `while (!q.empty())` menjadi inti penelusuran BFS yang beroperasi maksimal dalam kompleksitas waktu $O(R \times C)$ sebab setiap sel grid jalan hanya akan diproses dan mengokupasi antrean murni sebanyak satu kali, pada baris [33] syntax `if` di dalam iterasi arah secara ketat memvalidasi kelayakan langkah agar tidak menabrak batas tepi dimensi, menyeleksi sel yang murni bernilai jalan `0`, dan memastikan sel tersebut belum pernah

dikunjungi sebelumnya, pada baris [34 hingga 36] program memperbarui akumulasi jarak langkah *dist*, mendaftarkan koordinat baru tersebut ke antrean lanjutan, lalu langsung mencetak jarak optimum dan merampungkan fungsi melalui *return 0*; apabila koordinat target FR dan FC sudah terjamah, lalu pada baris [41 dan 42] syntax *cout << -1* serta *return 0*; akan dieksekusi secara pasif manakala perulangan antrean BFS telah usai dibongkar seluruhnya namun sel labirin tujuan tak kunjung teridentifikasi yang mengindikasikan ketiadaan rute keluar.

SOAL 4

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$SR\ SC$

$FR\ FC$

Keterangan:

- R adalah jumlah baris grid.

- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	14

A. Source Code

Tabel 4 Source Code prob4.cpp

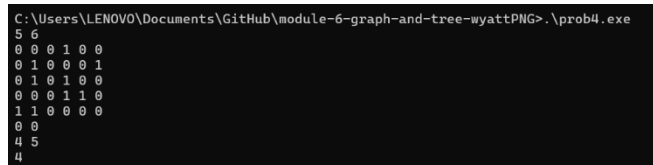
```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int R, C, FR, FC, count_path;
6  vector<vector<int>> grid;
7  vector<vector<bool>> visited;
8
9  int dr[] = {-1, 1, 0, 0};
10 int dc[] = {0, 0, -1, 1};
11
12 void dfs(int r, int c) {
13     if (r == FR && c == FC) { count_path++;
14     return; }
15     for (int d = 0; d < 4; d++) {
16         int nr = r + dr[d], nc = c + dc[d];
17         if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == 0 && !visited[nr][nc]) {
18             visited[nr][nc] = true;
```

```

18         dfs(nr, nc);
19         visited[nr][nc] = false;
20     }
21 }
22 }
23
24 int main() {
25     cin >> R >> C;
26     grid.assign(R, vector<int>(C));
27     for (int i = 0; i < R; i++)
28         for (int j = 0; j < C; j++)
29             cin >> grid[i][j];
30
31     int SR, SC;
32     cin >> SR >> SC >> FR >> FC;
33
34     count_path = 0;
35     visited.assign(R, vector<bool>(C, false));
36     visited[SR][SC] = true;
37     dfs(SR, SC);
38
39     cout << count_path << endl;
40     return 0;
41 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\prob4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Algoritma ini mengimplementasikan metode Depth-First Search (DFS) rekursif berpadu dengan teknik backtracking untuk mengeksplorasi dan menghitung keseluruhan kemungkinan rute sederhana yang valid dari titik awal menuju target di dalam labirin grid dua dimensi tanpa mengunjungi sel yang sama lebih dari satu kali pada satu siklus jalur. Pada baris [5 hingga 7] deklarasi variabel global seperti batas ukuran matriks, titik koordinat tujuan, *count_path*, struktur *grid*, dan penyimpanan jejak rute *visited* digunakan untuk menyediakan kerangka alokasi memori lintas fungsi yang mendominasi kompleksitas ruang secara keseluruhan sebesar $O(R \times C)$, pada baris [9 dan 10] pemetaan array *dr* dan *dc* menyimpan empat kemungkinan pergerakan geser ortogonal bernilai $O(1)$, pada baris [12] syntax *void dfs(int r, int c)* mendeklarasikan fungsi pencarian mendalam yang secara teoretis memiliki kompleksitas waktu laju asimtotik level eksponensial (hingga $O(3^{(R \times C)})$) mengingat sifat alamiah komputasinya yang membongkar semua kemungkinan percabangan yang valid, pada baris [13] syntax *if (r == FR && c == FC)* mengevaluasi pencapaian koordinat garis tujuan secara konstan $O(1)$ untuk menjumlahkan variabel pelacak *count_path* dan meredam penelusuran di titik tersebut menggunakan syntax *return;*, pada baris [14 hingga 16] perulangan *for* menelusuri penambahan koordinat arah baru sekaligus mengevaluasi keabsahan logisnya secara instan $O(1)$ guna memastikan batas koordinat murni menapak di dalam labirin, merupakan jalan kosong 0, serta sama sekali belum dilintasi dalam riwayat matriks *!visited*, pada baris [17 hingga 19] algoritma mengeksekusi inti manuver backtracking menggunakan syntax *visited[nr][nc] = true;* untuk menandai jejak kunjungan, memperdalam jangkauan rekursi ke koordinat tersebut melalui panggilan *dfs(nr, nc);*, dan yang terpenting secara absolut menggunakan syntax *visited[nr][nc] = false;* guna menghapus jejak memori pada sel yang sama seussai langkah rekursi selesai agar dapat dijajaki kembali secara utuh dari sudut kemungkinan percabangan jalan alternatif, pada baris [24 hingga 32] di dalam ruang lingkup utama *main()*, program membaca masukan spesifikasi dimensi beserta isian selang-seling matriks topografinya memakan ruang waktu berwujud $O(R \times C)$, pada baris [34 hingga 37] jumlah cacah lintasan disetel kembali ke nilai awal, pemetaan status perlintasan diinisialisasi menyita $O(R \times C)$, sel start ditutup dengan *true* agar tidak ditabrak berbalik oleh rutenya sendiri, kemudian fungsi *dfs(SR, SC);*

dicetuskan untuk mendobrak jalannya program simulasi pembongkaran, lalu diakhiri pada baris [39] dengan syntax *cout* guna merepresentasikan hasil tabulasi jumlah lintasan sah ke layer.

SOAL 5

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.

- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70
50 30 70 20 40 60 80	60 80
4	[Inorder] : 20 30 40 50 60 70
PREORDER	80
INORDER	[Postorder] : 20 40 30 60 80
POSTORDER	70 50
ALL	

	[Preorder] : 50 30 20 40 70 60 80 [Inorder] : 20 30 40 50 60 70 80 [Postorder] : 20 40 30 60 80 70 50
--	--

A. Source Code

Tabel 5 Source Code prob5.cpp

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include "RedBlackTree.h"
5  using namespace std;
6
7  void preorder(const RedBlackTree::Node* node,
8               vector<int>& result) {
9      if (node->isNil) return;
10     result.push_back(node->key);
11     preorder(node->left, result);
12     preorder(node->right, result);
13 }
14
15 void inorder(const RedBlackTree::Node* node,
16              vector<int>& result) {
17     if (node->isNil) return;
18     inorder(node->left, result);
19     result.push_back(node->key);
20     inorder(node->right, result);
21 }

```

```

21 void postorder(const RedBlackTree::Node* node,
vector<int>& result) {
22     if (node->isNil) return;
23     postorder(node->left, result);
24     postorder(node->right, result);
25     result.push_back(node->key);
26 }
27
28 void printTraversal(const string& label, const
RedBlackTree::Node* root) {
29     vector<int> res;
30     if (label == "[Preorder]  : ") preorder(root,
res);
31     else if (label == "[Inorder]      : ")
inorder(root, res);
32     else postorder(root, res);
33
34     cout << label;
35     for (int i = 0; i < (int)res.size(); i++) {
36         if (i) cout << " ";
37         cout << res[i];
38     }
39     cout << endl;
40 }
41
42 int main() {
43     int N;
44     cin >> N;
45
46     RedBlackTree rbt;
47     for (int i = 0; i < N; i++) {

```



```

48         int x; cin >> x;
49         if (!rbt.contains(x)) rbt.insert(x);
50     }
51
52     int Q;
53     cin >> Q;
54
55     vector<string> queries(Q);
56     for (int i = 0; i < Q; i++) cin >> queries[i];
57
58     for (int i = 0; i < Q; i++) {
59         if (rbt.empty()) {
60             cout << "Tree kosong. Tidak ada yang
bisa ditampilkan." << endl;
61             continue;
62         }
63
64         const string& query = queries[i];
65         if (query == "PREORDER") {
66             printTraversal("[Preorder]    : ",
rbt.root());
67         } else if (query == "INORDER") {
68             printTraversal("[Inorder]      : ",
rbt.root());
69         } else if (query == "POSTORDER") {
70             printTraversal("[Postorder]   : ",
rbt.root());
71         } else if (query == "ALL") {
72             printTraversal("[Preorder]    : ",
rbt.root());

```

```

73         printTraversal("[Inorder]      : ",
rbt.root());
74         printTraversal("[Postorder]   : ",
rbt.root());
75     }
76 }
77
78 return 0;
79 }

```

B. Output Program

```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>. \prob5.exe
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 5. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Pada baris [1 hingga 5] syntax `#include` mengimpor library standar C++ serta file header lokal `RedBlackTree.h` untuk menangani fungsi input-output, manipulasi string, instansiasi larik dinamis, dan mendefinisikan struktur pohon biner dengan kompleksitas memori $O(1)$, pada baris [7 hingga 26] blok fungsi `preorder`, `inorder`, dan `postorder` direpresentasikan secara rekursif murni untuk membaca dan menyalin data kunci (key) dari tiap simpul node ke dalam struktur penyimpanan vektor dengan memakan kompleksitas waktu operasional mutlak $O(N)$ serta kerumitan memori ruang senilai $O(N)$, pada baris [28 hingga 40] fungsi pendukung `printTraversal` menyeleksi metode rekursi penelusuran berdasar spesifikasi `label` yang diminta, yang disusul oleh perulangan `for` untuk menelusuri rentetan array hasil dan mencetaknya secara linier dengan waktu $O(N)$, pada baris [42 hingga 44] syntax `int main()` menginisiasi jalannya siklus program dan membaca batasan jumlah elemen bilangan yang diberikan, pada baris [46 hingga 50] deklarasi objek pohon `RedBlackTree rbt;` dilakukan di luar perulangan pemasukan

data yang mana syntax logis `if (!rbt.contains(x)) rbt.insert(x);` memblokir pengisian angka duplikat dan mengeksekusi penyisipan pada struktur pohon berkecepatan pencarian optimal sehingga merampungkan pembentukan memori berkapasitas $O(N)$ dengan laju komputasi agregat $O(N \log N)$, pada baris [52 hingga 56] syntax `cin >> Q` beserta alokasi array `vector<string> queries` bertugas membaca dan menyerap keseluruhan teks perintah cetak yang masuk dengan menyita kelonggaran memori sistem sebesar $O(Q)$, pada baris [58 hingga 76] perulangan `for` mengawali pergerakan untuk memroses daftar kueri secara berurutan, pada baris [59 hingga 62] syntax kondisional `if (rbt.empty())` memvalidasi fenomena kekosongan elemen data demi mencetak peringatan pohon kosong secara konstan $O(1)$, pada baris [64 hingga 75] syntax percabangan `if` serta `else if` menyeleksi tipe string perintah (seperti `PREORDER`, `INORDER`, `POSTORDER`, dan `ALL`) lalu mendelegasikan perintah cetak yang menyusur langsung dari akar utama objek (`rbt.root()`) dengan laju kompleksitas mencetak penuh sebesar $O(N)$ per kueri, lalu bermuara pada baris [78] di mana syntax `return 0;` menutup eksekusi skrip komputasi struktur data ini secara paripurna.

TAUTAN GIT

<https://github.com/DSA26-ULM/module-6-graph-and-tree-wyattPNG>